MODULE 5 — Memory Management

Senior SWE Interview Track — Operating Systems

5.1 Virtual Memory

1. Why Interviewers Ask This

The single most important OS abstraction. Interviewers use it to test whether you understand *why* your process sees 128 TB of address space on a 16 GB machine, and what RSS vs VSZ actually mean.

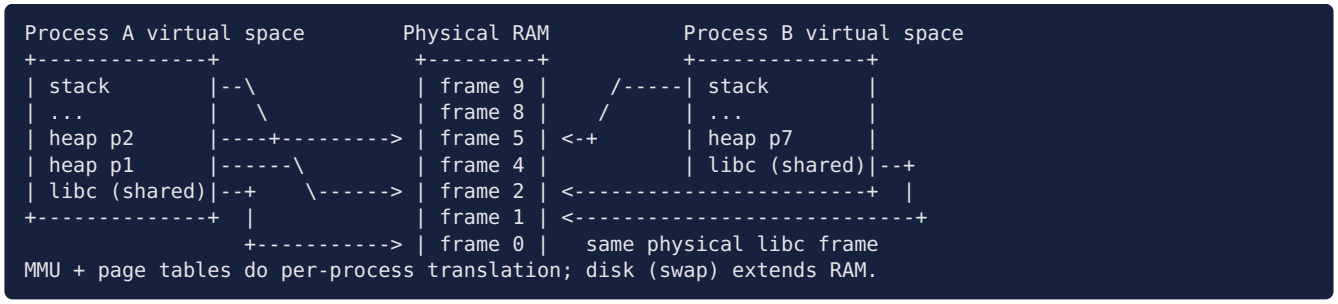
2. Core Concept

Virtual memory gives every process its own private address space; the MMU translates virtual → physical addresses per access. Benefits: **isolation** (processes can't touch each other), **overcommit** (allocate more than RAM; back with demand paging/swap), **sharing** (map the same physical page into many processes — libc, page cache), **contiguity illusion** (fragmented physical RAM looks contiguous).

3. Internal Working

- Address space carved into fixed **pages** (4 KB default); RAM into **frames**. Page tables map pages→frames with permission bits (R/W/X, user/kernel).
- CPU's MMU walks the page table on each access (cached by TLB). Invalid/missing mapping → **page fault** → kernel decides: load from disk (legit), COW copy, grow stack, or SIGSEGV (bug).
- Linux tracks a process's mappings as **VMAs** (see `/proc/<pid>/maps`); physical pages are allocated lazily on first touch.
- Overcommit: `malloc` reserves address space; RAM is committed on touch — why `VSZ` ≫ `RSS`, and why the **OOM killer** exists (the kernel's promise can outstrip reality).

4. ASCII Diagram



5. Real Production Example

- Redis `fork()`-based RDB snapshots rely on virtual memory + COW: the child "copies" 60 GB instantly.
- Containers on one host: each process believes it owns the address space; cgroups cap the physical side.
- JVM `-Xmx` reserves huge virtual ranges upfront; resident grows with the live heap.

6. Advantages

Isolation/security, simple linking model (fixed layout + ASLR), overcommit efficiency, memory-mapped files, sharing.

7. Trade-offs

Translation overhead (TLB pressure), page-fault latency cliffs, overcommit → OOM-killer surprises, an extra layer to reason about (RSS vs VSZ vs shared confuses monitoring).

8. Common Mistakes

- Reading `VSZ` as memory usage (JVMs "use" 20 GB VSZ with 2 GB RSS).
- "Virtual memory = swap" — swap is just one backing store; VM exists even with swap off.
- Not knowing that `malloc` success ≠ RAM available (overcommit; failure comes later as OOM kill on touch).

9. Performance Implications

TLB miss → page-table walk (up to 4 memory accesses on x86-64, ~100 ns); major fault → disk (µs on NVMe, ms on HDD). Random access over a working set larger than RAM = thrashing (Module 6).

10. Common Interview Questions

- "What happens, in full detail, when your program dereferences a pointer?"
- "RSS vs VSZ vs shared memory in `top` — explain."

11. Follow-up Questions

- "How can total RSS of all processes exceed RAM?" (shared pages counted per-process — see PSS)
- "What is memory overcommit and what's the failure mode?" (OOM killer; `vm.overcommit_memory` modes)

12. Coding/Debugging Scenarios

- Container OOM-killed though heap limit < container limit → RSS includes native/mmap/metaspace beyond the heap; measure the whole process.
- Latency spikes correlated with first access to a big `mmap`ed file → page-fault storms; pre-touch or `madvise(WILLNEED)`.

13. Best Practices

Monitor RSS/PSS not VSZ; set container limits with headroom over the true working set; understand your runtime's native overhead.

14. Practice Questions

1. Trace a load instruction: TLB hit path, TLB miss path, minor fault path, major fault path.
2. Two processes each report 1 GB RSS; the host shows 1.2 GB used. Explain.

5.2 Paging

1. Why Interviewers Ask This

The mechanism behind virtual memory. Tests address-splitting math and whether you know why fixed-size pages beat variable segments.

2. Core Concept

Divide virtual space into fixed **pages** and RAM into same-size **frames**; any page → any frame. Eliminates **external fragmentation** (all chunks same size); costs **internal fragmentation** (avg ~½ page waste per allocation tail).

3. Internal Working

Virtual address = [page number | offset]. 4 KB pages → offset = low 12 bits. Translation: page number → (page table) → frame number; physical = frame·4096 + offset. Permissions checked at the same time (present, R/W/X, U/K, accessed/dirty bits maintained by hardware).

4. ASCII Diagram

```
32-bit VA, 4KB pages:
| 20 bits: virtual page number (VPN) | 12 bits: offset |
VPN --page table--> PFN (physical frame number)
PA = PFN<<12 | offset

VA 0x00403ABC -> VPN 0x403, offset 0xABC
PT[0x403] = frame 0x1F2  =>  PA = 0x1F2ABC
```

5. Real Production Example

Everything on x86-64/ARM64. Databases and JVMs also use **huge pages** (2 MB/1 GB) to cut TLB pressure — e.g., Postgres `huge_pages=on`, JVM `-XX:+UseLargePages`.

6. Advantages

No external fragmentation; simple allocation (any free frame); enables sharing, COW, demand paging, per-page permissions.

7. Trade-offs

Internal fragmentation; page-table memory; every access needs translation (TLB saves it); page granularity means 1-byte-used still costs 4 KB resident.

8. Common Mistakes

- Mixing up internal (paging) vs external (segmentation/heap) fragmentation.
- Botching the math: 4 KB = 2^{12} → 12 offset bits; know 2^{10}=1K, 2^{20}=1M, 2^{30}=1G cold.

9. Performance Implications

Page size trade-off: bigger pages → fewer TLB misses, smaller tables, but more internal fragmentation and slower/coarser I/O; 4 KB default, 2 MB huge pages for big heaps (Transparent Huge Pages can help or hurt — khugepaged stalls, why some DBs say disable THP).

10–11. Interview & Follow-ups

- “64-bit VA, 4 KB pages: how many offset bits? How big would a flat page table be?” (leads to multi-level)
- “Why did huge pages help your database?” (TLB reach: 1536 entries × 4 KB = 6 MB vs × 2 MB = 3 GB)

12. Coding/Debugging Scenario

`perf stat -e dTLB-load-misses` high on a 100 GB in-memory store → enable explicit huge pages; verify miss-rate and p99 drop.

13. Best Practices

Align hot data structures to pages; consider huge pages past ~10 GB heaps; know your platform’s THP policy.

14. Practice Questions

1. 48-bit VA, 4 KB pages, 8-byte PTEs — size of a flat page table? (2^{36} PTEs × 8 B = 512 GB → motivates multi-level)
2. Compute PA for VA 0x7FFF12345 given a tiny page table you invent.

5.3 Segmentation

1. Why Interviewers Ask This

Mostly for contrast with paging and to explain the *meaning* of “segmentation fault”. Keep it brief and comparative.

2. Core Concept

Divide the address space into variable-length **segments** with semantic meaning (code, data, stack, heap), each with base + limit + permissions. Translation: segment base + offset, bounds-checked.

3. Internal Working

Segment table holds (base, limit, perms). VA = (segment selector, offset); hardware checks offset < limit then adds base. Variable sizes → **external fragmentation** (free RAM in useless gaps) → compaction needed. x86 had real segmentation; x86-64 effectively flattens it (bases 0) and uses paging — %fs/%gs survive for TLS.

4. ASCII Diagram

```
Segment table:              Physical memory:
code:  base 0x1000 lim 4K   |code (4K)|  gap  |heap(16K)| gap |stack(8K)|
heap:  base 0x8000 lim 16K        ^ external fragmentation ^
stack: base 0xF000 lim 8K
VA (heap, 0x123) -> check 0x123 < 16K -> PA 0x8123
Offset >= limit  -> segmentation fault (the historical name)
```

5. Real Production Example

Modern relevance: the *concept* survives as VMAs/memory regions (`/proc/maps` shows code/heap/stack/mmap regions with permissions — "paged segmentation" in spirit); "segfault" = access outside any valid region.

6. Advantages

Semantic protection units; natural sharing of whole segments; grows to fit logical structure.

7. Trade-offs

External fragmentation + compaction; complex allocation (best/first fit); segments limited in number.

8. Common Mistakes

- Claiming modern x86-64 "uses segmentation" for isolation (it's paging; segmentation is vestigial).
- Not knowing why the fault is *named* segmentation fault.

9. Performance Implications

Historical; the enduring lesson is variable-size allocation → external fragmentation — the same math that governs heap allocators (5.9).

10–11. Interview & Follow-ups

- "Paging vs segmentation — table of differences." "What actually happens on a segfault today?" (page-level permission/present violation → SIGSEGV)

12. Coding/Debugging Scenario

`cat /proc/<pid>/maps` of a crashed process + the faulting address from `dmesg` → determine which region boundary was violated (NULL page? stack guard? freed mmap?).

13. Best Practices

Use the paging-vs-segmentation contrast to *show* you understand fragmentation types — that's its interview job.

14. Practice Questions

1. Fill the table: fragmentation type, size unit, protection granularity, modern usage — paging vs segmentation.
2. Why does the kernel place a guard gap below the stack VMA?

5.4 TLB (Translation Lookaside Buffer)

1. Why Interviewers Ask This

The TLB is why virtual memory is *fast enough to be free-ish*, and TLB reasoning explains real performance cliffs (context switches, huge pages, random access).

2. Core Concept

The TLB is a small, very fast cache of recent VPN→PFN translations inside the MMU. Hit: translation in <1 cycle. Miss: hardware walks the page table (~tens–100+ ns), then caches the entry.

3. Internal Working

- Typical: L1 dTLB/iTLB 64–128 entries, L2 STLB ~1.5–2K entries, per core.
- **Reach**: 1536 entries × 4 KB ≈ 6 MB — touch more than that randomly and you miss constantly; 2 MB huge pages raise reach to ~3 GB.
- Context switch to another address space: TLB entries invalid → flush, unless entries are tagged (**PCID/ASID**) — the hidden cost of process switches (Module 1).
- **TLB shootdown**: when one core changes a mapping (munmap, COW break, page migration), it must IPI all cores that may cache the entry — expensive, scales badly with core count (why frequent mmap/munmap hurts multithreaded services).

4. ASCII Diagram

```
CPU: VA -> [TLB lookup]
        hit (99%+): PA in <1 cycle
        miss: [HW page walk: PGD->PUD->PMD->PTE, up to 4 mem reads]
              -> install in TLB -> PA
Reach: 4KB pages: ~6MB   |   2MB huge pages: ~3GB
Shootdown: core0 munmap -> IPI-> core1..N flush entry (stalls all)
```

5. Real Production Example

- Databases enabling huge pages for buffer pools (Postgres, Oracle) — measured double-digit gains on TLB-bound workloads.
- Hash-join/random-pointer-chasing analytics: TLB misses dominate; sorted/partitioned access patterns fix it.
- glibc `MALLOC_ARENA` churn or allocators doing frequent `munmap` → shootdown storms in many-threaded services.

6. Advantages

Makes paging ~free for hot working sets; transparent.

7. Trade-offs

Limited reach; flush costs on switches; shootdowns serialize cores; another layer of “works until it cliffs”.

8. Common Mistakes

- Explaining paging with zero mention of the TLB (translation would be 2–5× slowdowns otherwise).
- Not knowing shootdowns exist (senior differentiator).

9. Performance Implications

Random access across >TLB-reach memory: every access ≈ +100 ns walk → can halve throughput. Measure with `perf stat -e dTLB-load-misses,dtlb_load_misses.walk_duration`.

10–11. Interview & Follow-ups

- “What makes virtual memory fast despite per-access translation?” “Why do huge pages help?” “What is a TLB shootdown and when does it bite?”

12. Coding/Debugging Scenario

In-memory graph traversal 3× slower than back-of-envelope: perf shows 30% cycles in page walks → switch node pool to 2 MB huge pages; misses collapse.

13. Best Practices

Favor locality (arrays over pointer soup); huge pages for large hot heaps; avoid mmap/munmap churn on hot paths (pool your mappings).

14. Practice Questions

1. Compute TLB reach for 1024 entries with 4 KB vs 2 MB pages; relate to a 50 GB working set.
2. Why does a process context switch cost more than the register save/restore suggests? (TLB/cache refill)

5.5 Page Tables & 5.6 Multi-Level Page Tables

1. Why Interviewers Ask This

“Why multi-level?” is a top-5 OS interview question with a beautiful quantitative answer (sparse address spaces), and the 4-level walk is the standard “explain translation end-to-end” test.

2. Core Concept

- Page table: per-process map VPN→PFN + flags (present, R/W, U/K, NX, accessed, dirty).
- Flat table for 48-bit VA at 4 KB pages = 2^{36} entries × 8 B = **512 GB per process** — impossible.
- **Multi-level** tables are a radix tree: allocate only the branches that map something. Sparse address spaces (typical: few hundred MB used of 128 TB) → tables cost only ~MBs.

3. Internal Working

x86-64 4-level (PGD→PUD→PMD→PTE): 48-bit VA = 9+9+9+9 index bits + 12 offset. CR3 holds the root’s physical address. Hardware walker: CR3 → PGD[i1] → PUD[i2] → PMD[i3] → PTE[i4] → frame. Any non-present entry at any level cuts the whole subtree (unmapped 512 GB costs one empty PGD slot). PMD-level “huge” entry = 2 MB page (skips one level). 5-level paging (57-bit VA) exists for huge-memory machines. Each level’s tables are 4 KB (512 × 8 B entries).

4. ASCII Diagram

```
VA (48-bit): | 9b PGD | 9b PUD | 9b PMD | 9b PTE | 12b offset |
CR3 -> [PGD] --i1--> [PUD] --i2--> [PMD] --i3--> [PTE] --i4--> frame
         512e          512e          512e          512e
Sparse win: unmapped region => NULL at top level, no lower tables exist.
Cost of a TLB miss: up to 4 dependent memory reads (~100+ ns).
PTE flags: [P|RW|US|A|D|NX| PFN.....]
```

5. Real Production Example

Every Linux/Windows process. Fork copies page-table structure (not data pages — COW); `fork()` of a huge-RSS Redis is dominated by copying these tables (a 60 GB RSS ≈ ~120 MB of page tables to copy → tens of ms pause).

6. Advantages

Memory ∝ *mapped* space, not address-space size; hierarchical permissions; hardware-walkable; natural huge-page support.

7. Trade-offs

Walk latency multiplies at each level (mitigated by TLB + partial-walk caches); table updates need TLB shootdowns; deep trees complicate the kernel’s mm code.

8. Common Mistakes

- Can't do the 512 GB flat-table math (memorize: 48-bit, 4 KB, 8 B PTE ⇒ $2^{36} \times 8$ = 512 GB).
- Saying page tables live in the process's virtual memory (they're kernel-managed physical structures; the process never sees them).
- Forgetting the *dirty/accessed* bits — they drive page replacement (Module 6).

9. Performance Implications

TLB miss cost = up to 4 (or 5) dependent loads; huge pages remove one level *and* multiply reach; fork cost scales with mapped memory (why big processes prefer `posix_spawn`/vfork semantics).

10. Common Interview Questions

- "Why multi-level page tables? Do the math." "Walk me through translating one address on x86-64."

11. Follow-up Questions

- "What are inverted page tables?" (global frame→(pid,vpn) hash — memory ∝ RAM not VA; used by PowerPC historically)
- "How does the kernel find which processes map a physical page?" (reverse mapping — rmap; needed for eviction)

12. Coding/Debugging Scenarios

- Redis latency spike every BGSAVE → fork page-table copy; mitigations: huge pages reduce PTE count (careful: THP+fork COW amplification!), or diskless replication.

13. Best Practices

Know your per-level math cold; mention accessed/dirty bits and shootdowns to show depth.

14. Practice Questions

1. For VA 0x00007F1234ABC987, compute the four 9-bit indices (hex ok).
2. A process maps 1 TB contiguously with 4 KB pages: how much page-table memory? (~2 GB PTEs + ~4 MB PMDs + …) With 2 MB pages? (~4 MB)

5.7 Demand Paging

1. Why Interviewers Ask This

Explains program startup, `mmap`, lazy allocation, and the minor/major fault distinction that shows up in every perf investigation.

2. Core Concept

Pages are loaded/allocated **only on first access**, not upfront. `exec` maps the binary without reading it; `malloc` returns address space without RAM; touching a page triggers a fault that materializes it.

3. Internal Working

1. Access to non-present page → CPU raises page fault → kernel handler.
2. Kernel checks the VMA: valid mapping? - **Minor fault**: page already in RAM (page cache, zero page, COW) — just map it (~µs). - **Major fault**: read from disk (binary, swap, mapped file) — schedule I/O, block the task (µs–ms). - No valid VMA / bad perms → SIGSEGV.
3. Update PTE, resume the instruction transparently. Optimizations: readahead (fault-around), zero-page sharing for untouched anonymous memory.

4. ASCII Diagram

```
touch page --> fault --> VMA valid?
                         |-- no --> SIGSEGV (bug)
                         |-- yes, in page cache --> minor fault: map (fast)
                         |-- yes, on disk/swap  --> major fault: I/O, block (slow)
Startup of a 2GB binary: maps in ms; pages fault in as code paths execute.
```

5. Real Production Example

- Fast container/process cold starts (only touched pages load).
- `ps -o min_flt,maj_flt`; `sar -B` majflt/s as a thrashing signal.
- ML serving: first inference is slow because weights fault in from disk — fixed by pre-touch/`mlock`/warmup requests.

6. Advantages

Fast startup, memory efficiency (unused features never load), enables overcommit and huge sparse mappings.

7. Trade-offs

First-touch latency cliffs; unpredictable performance until warm; page-fault storms under memory pressure (thrashing).

8. Common Mistakes

- Not distinguishing minor vs major faults (interviewers probe this exactly).
- Treating page faults as errors — most are the design working as intended.

9. Performance Implications

Minor ~1–5 µs; major ~100 µs (NVMe) to ~10 ms (HDD). p99-sensitive services `mlock` hot data or pre-touch after deploy; majflt/s > ~0 sustained on a latency-critical box is an incident.

10–11. Interview & Follow-ups

- “What happens on `malloc(1GB)` — when is RAM actually used?” “Minor vs major fault?” “Why was the first request after deploy slow?”

12. Coding/Debugging Scenario

Post-deploy p99 spike for 2 minutes: `sar -B` shows major faults → warm-up: sequentially touch the model/index files (`vmtouch`, `madvise(WILLNEED)`), or `mlock`.

13. Best Practices

Warm critical paths before taking traffic; `mlockall` for hard-latency processes (with care); watch majflt/s per service.

14. Practice Questions

1. `memset(malloc(1<<30), 0, 1<<30)`: describe every fault and when RSS grows.
2. Why can reading a fresh `mmap`ed file be slower than `read()` for a single sequential pass? (per-page faults vs batched copies; readahead interplay)

5.8 Copy-on-Write (COW)

1. Why Interviewers Ask This

“How does fork() copy 60 GB instantly?” is a top interview question, and COW connects fork, Redis, and memory spikes into one story.

2. Core Concept

Instead of copying memory at `fork()`, parent and child **share all pages read-only**; the first write by either side faults, and the kernel copies **only that page** (then makes both writable). Copy cost is deferred and proportional to *pages actually written*.

3. Internal Working

1. `fork()`: copy page *tables*, mark both sides' PTEs read-only, bump page refcounts.
2. Write → protection fault → kernel sees COW mapping → allocate frame, `memcpy` 4 KB, remap writer's PTE writable, decrement refcount (last owner just gets it back writable — no copy).
3. TLB shootdown for the changed PTE. Also used for: `mmap(MAP_PRIVATE)` file mappings, KSM (same-content page merging), zygote-style app forking (Android), and the zero page.

4. ASCII Diagram

```
fork():
Parent PTEs -----> [frame X (RO)] <----- Child PTEs   refcount=2
Child writes page:
  fault -> copy X -> X' ; child PTE -> X' (RW); parent PTE -> X (RW when last)
Memory cost after fork = pages *written*, not total size.
Redis 60GB fork: instant; but heavy writes during BGSAVE => up to 2x RAM!
```

5. Real Production Example

- **Redis BGSAVE/AOF-rewrite**: fork child snapshots memory; COW keeps it cheap — but a write-heavy period during the save can double resident memory (classic prod incident + interview scenario).
- Android Zygote: preloaded framework pages shared COW across all app processes.
- `fork+exec` for shells/servers: near-zero copy since exec replaces the space anyway.

6. Advantages

O(page-tables) fork; memory dedup; snapshot semantics for free; enables fork-based concurrency patterns.

7. Trade-offs

Write-fault latency blips; memory usage becomes workload-dependent and *spiky* (COW break storms); refcounting/rmap complexity; THP amplifies COW cost (2 MB copies).

8. Common Mistakes

- "Fork copies all memory" (pre-1980s answer).
- Forgetting the worst case: fork + write-everything = 2× memory → OOM (must size Redis boxes for it).
- Missing that only the *writer's* side copies; reader keeps the original.

9. Performance Implications

COW break = fault + 4 KB copy + shootdown (~µs). A snapshotting child pins old pages: long saves + hot writes = memory balloon; monitor `fork` duration and RSS during BGSAVE.

10–11. Interview & Follow-ups

- "Explain how Redis snapshots 60 GB without doubling memory — and when it *does* double."
- "What exactly is copied at fork time?" (page tables + task metadata) "How does COW interact with huge pages?"

12. Coding/Debugging Scenario

Redis OOM-killed during BGSAVE on a 64 GB box with 40 GB dataset → COW inflation under write load; fixes: replica-based persistence, more headroom, `vm.overcommit_memory=1` (Redis's own recommendation), schedule saves off-peak.

13. Best Practices

Budget worst-case COW memory for fork-snapshot systems; keep snapshot windows short; prefer `posix_spawn` for spawn-only use.

14. Practice Questions

1. Parent has 10 GB RSS, forks, child reads everything, parent rewrites 1 GB: final memory footprint?
2. Why is `vfork`/`posix_spawn` still faster than COW fork? (skips even page-table copy)

5.9 Memory Allocation (malloc & friends)

1. Why Interviewers Ask This

Bridges OS and everyday engineering: “what happens on malloc?” tests the user-allocator vs kernel boundary; fragmentation questions test long-running-service judgment.

2. Core Concept

Two layers: the **kernel** hands out address space in pages (`brk/sbrk` for the heap end, `mmap` for large blocks); the **user-space allocator** (glibc ptmalloc, jemalloc, tcmalloc, mimalloc) slices pages into objects with free lists/size classes. `malloc` is a library call, *not* a syscall (usually).

3. Internal Working

- Small allocs: size-class bins + per-thread caches (tcache/arenas) → no lock on the fast path, ~10–20 ns.
- Large allocs (glibc ≥ 128 KB default): direct `mmap`, returned to OS on free.
- `free` returns to bins; heap top can shrink via `brk` only if the top is free → **freed ≠ returned to OS** (RSS stays high — the classic “leak that isn’t”).
- Fragmentation: **external** (free space split into unusable gaps — allocator-level now, since paging fixed it at the physical level) and **internal** (size-class rounding waste).
- Kernel side on first touch: page fault → zeroed frame (demand paging as above).

4. ASCII Diagram

```
malloc(64B):
  thread cache bin[64] pop --> pointer (ns, no syscall)
  bin empty -> refill from central arena -> maybe mmap/brk new pages
malloc(10MB):
  mmap anonymous pages --> returned directly; freed via munmap

Heap fragmentation:
|used|free 24B|used|free 40B|used|   malloc(48B) FAILS to fit gaps
  -> allocator grows heap despite 64B "free" (external fragmentation)
```

5. Real Production Example

- Long-running services switching glibc→jemalloc/tcmalloc for fragmentation and multi-thread scaling (Redis ships jemalloc; Meta/Google run their allocators fleet-wide).
- “RSS climbs forever but heap profiler shows constant live set” → fragmentation + free-but-not-returned pages; jemalloc `background_thread`/`muzzy_decay` tuning.

6. Advantages

User-space fast paths avoid syscalls; size classes bound external fragmentation; per-thread caches scale.

7. Trade-offs

Size classes waste memory (internal frag up to ~25%); per-thread caches hold memory hostage; allocator behavior varies wildly across implementations (portability of performance).

8. Common Mistakes

- “malloc is a system call.”
- Interpreting stable-RSS-after-free as a leak; or actual leaks as “fragmentation”.
- Ignoring allocation in hot loops (allocation is cheap; *cache misses and contention it causes* are not).

9. Performance Implications

Fast path ~ns; arena contention with many threads (glibc arenas vs tcmalloc per-CPU caches); huge-alloc mmap/munmap churn → page faults + TLB shootdowns (pool them).

10–11. Interview & Follow-ups

- “Walk through `malloc(100)` end-to-end, including when the kernel gets involved.”
- “Why doesn’t RSS drop after free?” “brk vs mmap allocations?” “Why jemalloc?”

12. Coding/Debugging Scenario

RSS grows 1%/day, heap profiler flat → fragmentation: check allocator stats (`malloc_stats`, `je_malloc_stats_print`); fix with jemalloc, arena caps, or object pooling for the offending size mix.

13. Best Practices

Pool/reuse hot-path objects; pick a modern allocator for multithreaded services; expose allocator stats in metrics; avoid mixed-lifetime allocations sharing regions (fragmentation fuel).

14. Practice Questions

1. Explain each transition: malloc(100) → tcache → arena → mmap/brk → page fault → zeroed frame.
2. Design an object pool for 4 KB request buffers — what fragmentation/waste do you eliminate?

5.10 Heap vs Stack

1. Why Interviewers Ask This

Deceptively basic; seniors are graded on the *why*: allocation cost mechanics, lifetime rules, overflow behavior, thread interactions.

2. Core Concept

- **Stack**: per-thread LIFO region for frames (locals, args, return addresses). Allocation = bump SP (one instruction); free = automatic at return. Fixed max size (default 8 MB Linux threads).
- **Heap**: process-wide region for dynamic, arbitrary-lifetime data via allocator. Slower, must be managed (free/GC), but sized by RAM not by a per-thread cap.

3. Internal Working

- Stack grows downward; guard page below → overflow = SIGSEGV on guard touch (usually). Frames: saved return address, saved regs, locals — why stack smashing is a security class (canaries, NX).
- Heap: allocator on top of brk/mmap (5.9). Escape analysis (Java/Go) moves would-be heap objects to the stack; Go stacks are growable (start ~2–8 KB, copied on growth) — contrast with fixed pthread stacks.

4. ASCII Diagram

```
High addr  +----------------+
           | stack T1 (8MB) |  grows v   [guard page]
           | stack T2 (8MB) |  grows v   [guard page]
           |   ...mmap...   |
           | heap           |  grows ^ via allocator
           | bss / data     |
Low addr   | code (RO,X)    |
           +----------------+
Stack alloc: sub rsp, N  (1 insn)   Heap alloc: allocator logic (ns-µs)
```

5. Real Production Example

- Stack overflow from deep recursion (JSON parsers on hostile input!) → crash; iterative rewrite or bigger stack.
- 10k-thread server: 10k × 8 MB virtual stacks (fine) but touched pages accumulate → tune stack size down.
- GC languages: allocation-rate-driven GC pauses → reduce heap churn (object reuse), rely on escape analysis.

6. Advantages

Stack: fastest possible alloc/free, cache-hot, no fragmentation, automatic lifetime. Heap: flexible size/lifetime, shareable across threads/functions.

7. Trade-offs

Stack: fixed limit, frame lifetime only (dangling pointers if escaped!), per-thread cost. Heap: allocator overhead, fragmentation, leaks/GC pressure, synchronization in allocator.

8. Common Mistakes

- Returning a pointer to a stack local (C/C++ classic).
- “Stack is in the CPU / heap is in RAM” — both are ordinary RAM regions.
- Not knowing default stack sizes or that thread stacks are just mmaps.

9. Performance Implications

Stack alloc ~0 cost and prefetch-friendly; heap alloc ~ns plus later cache misses; allocation-heavy hot paths often gain 2–10× from stack/arena conversion; GC cost ∝ allocation rate more than heap size (generational).

10–11. Interview & Follow-ups

- “Where do locals/globals/`new` objects/string literals live?” “What happens physically on stack overflow?” “How do goroutine stacks differ from pthread stacks?” (growable, movable → why C interop needs care)

12. Coding/Debugging Scenario

SIGSEGV with fault address just below the stack VMA → stack overflow, not a wild pointer; confirm via core dump RSP vs stack bounds; fix recursion or `ulimit -s` / `pthread_attr_setstacksize`.

13. Best Practices

Prefer stack/value types on hot paths; bound recursion on untrusted input; size thread stacks intentionally at high thread counts; profile allocation rate, not just heap size.

14. Practice Questions

1. For a C program: classify 8 variables (global, static local, local, malloc’d, literal, argv) into segments.
2. Why is `alloca`/VLA dangerous in servers? (unbounded stack use, no failure signal until SIGSEGV)

Module 5 Cheat Sheet (one page)

Numbers to know: page 4 KB (2^{12}); x86-64 4-level walk (9|9|9|9|12); flat table would be 512 GB; TLB ~1.5K entries → ~6 MB reach (4 KB) / ~3 GB (2 MB pages); minor fault µs, major fault 100 µs–10 ms; default pthread stack 8 MB.

Concept	One-liner	Interview hook
Virtual memory	Private per-process space, MMU-translated	RSS vs VSZ; overcommit → OOM killer
Paging	Fixed 4 KB pages ↔ frames	kills external frag, keeps internal
Segmentation	Variable semantic regions	external frag; why “segfault” is named so
TLB	Translation cache	reach math; shootdowns; PCID
Multi-level PT	Radix tree, pay for what you map	do the 512 GB math
Demand paging	Materialize on first touch	minor vs major faults; warmup
COW	Share RO at fork, copy on write	Redis BGSAVE 2× memory risk
malloc	Library size-classes over brk/mmap	freed ≠ returned; fragmentation vs leak
Stack vs heap	Bump-SP vs allocator	overflow mechanics; escape analysis

Fault decision tree: no VMA → SIGSEGV · in page cache/zero/COW → minor · on disk/swap → major.

Top Interview Questions

1. What happens when you dereference a pointer — full path including TLB, walk, faults.
2. Why multi-level page tables? (do the math) Walk one translation.
3. How does fork copy 60 GB instantly, and when does Redis double its memory?
4. malloc end-to-end; why doesn’t RSS drop after free?
5. Minor vs major page fault; why was the first request slow?
6. Huge pages: why do they help (TLB reach) and when do they hurt (THP stalls, COW amplification)?
7. Stack vs heap: allocation mechanics, overflow, lifetimes.

Common Mistakes (module-wide)

- VSZ as usage; “virtual memory = swap”; “malloc is a syscall”.
- Explaining translation without the TLB; missing dirty/accessed bits.
- Fragmentation vs leak confusion; forgetting COW’s 2× worst case.
- No minor/major fault distinction; returning stack pointers.

Mock Interview (self-test, ~25 min)

1. (Depth) From `mov rax,[rbx]` to data: enumerate every hardware/OS step for TLB hit, TLB miss, minor fault, major fault, and segfault paths.
2. (Math) 48-bit VA, 4 KB pages, 8 B PTEs: flat table size; then page-table memory for a process mapping 8 GB densely; then with 2 MB pages.
3. (Prod) Redis 45 GB on a 64 GB host is OOM-killed at 03:00 daily during BGSAVE. Explain mechanism and give three fixes with trade-offs.
4. (Prod) A service’s RSS grows forever; heap profiler says live set flat. Give your differential diagnosis tree (fragmentation, freed-not-returned, native/mmap leaks, page cache accounting) and the measurement for each.
5. (Trap) “We disabled swap, so page faults can’t happen.” Correct them precisely (file-backed pages, demand paging, page-cache eviction still fault).